LAB 4 — Distributed Transactions (2PC / 3PC)

Distributed Computing — AWS EC2-based

Estimated time: 3 hours
Work mode: Individual
Submission: Live demo or report + code repository
Cluster size: 3–5 nodes (1 coordinator + 2–4 participants)

---

1. Lab Objectives

By completing this lab, students will be able to:

- Explain the **atomicity problem** in distributed transactions.
- Implement **Two-Phase Commit (2PC)** and understand its limitations.
- Implement **Three-Phase Commit (3PC)** and compare it to 2PC.
- Observe **blocking behavior** and failure scenarios.
- Deploy and evaluate transaction protocols on **EC2 multi-node systems**.

This lab directly supports CLOs related to:

- distributed coordination,
- fault tolerance and recovery,
- consistency and atomicity,
- protocol analysis and comparison.

---

2. What You Will Build

You will build a **distributed transaction system** consisting of:

- **1 Coordinator**
- **2–4 Participants (Resource Managers)**

Each transaction:

- updates a logical resource (e.g., account balance, key-value entry),
- must either **commit on all nodes** or **abort everywhere**.

---

3. System Model

- **Nodes:** 3–5 EC2 instances

- **Roles:**
    - Coordinator
    - Participants
- **Communication:** Message passing (HTTP or TCP)
- **Failure model:** Crash-stop failures
- **Storage:** In-memory + optional write-ahead log

---

4. Transaction Model

Each transaction has a unique ID:

```
TX123
```

Example operation:

```
TRANSFER x = x - 10
```

Participants must:

- vote YES / NO,
- commit or abort based on coordinator decision.

---

5. Protocols to Implement

Part A — Two-Phase Commit

Phase 1 — Prepare

Coordinator:

```
PREPARE(TX)
```

Participants:

```
VOTE-YES / VOTE-NO
```

Phase 2 — Commit / Abort

- If all votes are YES → GLOBAL-COMMIT
- Else → GLOBAL-ABORT

Part B — Three-Phase Commit

3PC phases:

1. CanCommit
2. PreCommit
3. DoCommit

Goal:

- Avoid blocking when coordinator fails.

---

6. Node State (Required)

Each participant must maintain:

- Transaction state:
    - INIT
    - READY
    - COMMITTED
    - ABORTED
- Local resource state
- Write-Ahead Log (optional but recommended)

Coordinator must maintain:

- Active transaction table
- Votes from participants

---

7. Part 1 — Implement 2PC

Required Behavior

- Coordinator initiates transaction.
- Participants validate locally.
- Participants reply with votes.
- Coordinator decides commit/abort.
- Decision is propagated to all participants.

Logging Requirements

```
[Coordinator] TX123 PREPARE

[Node B] TX123 VOTE-YES
```

```
[Coordinator] TX123 GLOBAL-COMMIT
[Node C] TX123 COMMIT
```

---

8. Part 2 — Failure Scenarios

Demonstrate **at least one** of the following:

Scenario A — Coordinator Crash (Blocking)

1. Coordinator sends PREPARE.
2. Participants vote YES.
3. Coordinator crashes before decision.
4. Participants block (2PC limitation).

Scenario B — Participant Crash

1. Participant crashes before voting.
2. Coordinator times out.
3. Transaction aborts.

---

9. Part 3 — Implement 3PC

Extend your implementation to show:

- Non-blocking behavior when coordinator crashes
- Safe commit decision under bounded delays

Required Demo

- Show how 3PC avoids indefinite blocking.

---

10. Deployment Instructions (Example)

Coordinator

```
python3 coordinator.py --id C --port 8000 --participants B,D,E
```

Participants

```
python3 participant.py --id B --port 8001
python3 participant.py --id D --port 8002
```

```
python3 participant.py --id E --port 8003
```

---

11. Client Interaction

Trigger transaction:

```
python3 client.py transfer TX123 x -10
```

---

12. Deliverables

🟦 **Deliverable 1— Live Demo (3–4 minutes) with written report**

Report must show:

- Transaction start
- Voting phase
- Commit or abort
- Failure scenario
- Explanation of blocking / non-blocking behavior

🟦 **Deliverable 2 — Code Repository**

Must include:

- Coordinator and participant implementations
- Client or trigger scripts
- Minimal README (run instructions only)

---

13. Evaluation Criteria (100 points)

Criterion	Points
Correct 2PC implementation	30
Failure scenario demonstrated	20
Clear logs and states	10
Correct 3PC implementation	30
Explanation of limitations / comparison	10

14. Common Mistakes to Avoid

- Committing without unanimous votes
- Forgetting to handle timeouts
- No clear transaction IDs
- Ignoring blocking behavior
- Hardcoding outcomes